Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программная инженерия»

Лабораторная работа № 3

по дисциплине «Анализ алгоритмов»

Тема Рекурсивные и нерекурсивные алгоритмы

Студент Понарин А. А.

Группа ИУ7-55Б

Преподаватели Волкова Л. Л., Строганов Д. В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Аналитическая часть **4**

1.1 Постановка задачи 4

1.2 Операционное и информационное отношения 4

1.3 Четыре графовые модели 5

1.4 Условия параллельного выполнения 6

2 Конструкторская часть **7**

2.1 Нерекурсивный алгоритм 7

2.2 Рекурсивный алгоритм 8

2.3 Обоснование корректности и завершения 9

2.4 Требования к программе 9

3 Технологическая часть **10**

3.1 Средства реализации и структура программы 10

3.2 Реализация алгоритмов 10

3.3 Тестирование реализаций 11

3.4 Пример работы и воспроизведение 13

4 Исследовательская часть **14**

4.1 Уровень детализации графовых моделей 14

4.2 Граф управления нерекурсивного алгоритма 15

4.3 Информационный граф нерекурсивного алгоритма 16

4.4 Операционная история нерекурсивного алгоритма 17

4.5 Информационная история нерекурсивного алгоритма 18

4.6 Вершины моделей рекурсивного алгоритма 19

4.7 Граф управления рекурсивного алгоритма 20

4.8 Информационный граф рекурсивного алгоритма 21

4.9 Операционная история рекурсивного алгоритма 22

4.10 Информационная история рекурсивного алгоритма 23

4.11 Возможности параллельного выполнения исходного кода . . . 24

4.12 Сравнение графовых моделей и ресурсов 25

ЗАКЛЮЧЕНИЕ **27**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **28**

ВВЕДЕНИЕ

Цель работы — провести сравнение рекурсивного и нерекурсивного алгоритмов на основе графовых моделей алгоритмов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. дать определения четырёх графовых моделей алгоритмов, операционного и информационного отношений;
2. разработать нерекурсивный и рекурсивный последовательные алгоритмы разворота числа согласно варианту U;
3. реализовать разработанные алгоритмы на языке C++;
4. провести тестирование реализаций алгоритмов;
5. описать каждый алгоритм четырьмя графовыми моделями;
6. обосновать, какие фрагменты алгоритмов или их реализаций могут выполняться параллельно, а какие должны выполняться последовательно.

Реализация и тестирование рассматриваются в третьей части отчёта, графовые модели и возможности параллельного выполнения — в четвёртой. Оформление принято по ГОСТ 7.32–2017 [1] с сохранением структуры предоставленного образца; схемы алгоритмов выполнены с использованием обозначений ГОСТ 19.701–90 [2].

1 Аналитическая часть

1.1 Постановка задачи

Вариант U: дано натуральное число n, десятичная запись которого не содержит нулей. Требуется получить целое число, записанное теми же цифрами в противоположном порядке. Например, при $n = 179$ результат равен 971. Функция должна возвращать число целиком; вывод отдельных цифр вместо формирования результата не допускается.

В рекурсивном решении нельзя использовать циклы, строки, списки, массивы и срезы. Допустимы рекурсивные вызовы и целочисленная арифметика. Для выполнения общей задачи сравнения дополнительно разрабатывается нерекурсивный алгоритм с циклом. Таким образом, ограничение варианта соблюдается рекурсивным решением, а итеративное решение служит требуемым объектом сравнения.

Пусть

$$n = \sum_{i=0}^{k-1} c_i 10^i, \qquad c_i \in \{1, \dots, 9\}. \tag{1.1}$$

Искомая функция имеет вид

$$\mathrm{rev}(n) = \sum_{i=0}^{k-1} c_i 10^{k-1-i}. \tag{1.2}$$

В реализации принимаются числа от 1 до $10^{19}-1$ без нулевых цифр. Это ограничение машинного представления: все такие числа и их развороты помещаются в 64-разрядный беззнаковый целый тип. Отрицательные числа, ноль и числа с нулевыми цифрами не принадлежат области определения.

1.2 Операционное и информационное отношения

Рассматривается последовательное выполнение программы. *Срабатывание оператора* — одно конкретное исполнение оператора; один оператор цикла или рекурсивной функции может срабатывать многократно.

Операционное отношение связывает два срабатывания, если второе исполняется непосредственно после первого в выбранном последовательном выполнении. Оно описывает порядок исполнения, включая вычисление

условий и передачу управления.

Информационное отношение связывает срабатывание, выработавшее значение, со срабатыванием, которое использует именно это значение как операнд. Между записью и чтением не должно быть другой записи, заменившей передаваемое значение. Это прямая зависимость по данным, а не всякая последовательность действий с одинаковым именем переменной.

При переходе от срабатываний к операторам связь считается возможной, если она возникает хотя бы в одном допустимом выполнении. Для рекурсии различаются передача аргументов и получение результата: результат вызова становится доступен после возврата из соответствующего дочернего вызова.

1.3 Четыре графовые модели

Используются две компактные модели и две истории конкретного выполнения [3, 4]. Графы ориентированные; стрелка направлена от предшествующего действия к последующему либо от производителя значения к потребителю.

Граф управления содержит вершину для каждого оператора, включая условия. Дуга обозначает возможное непосредственное следование операторов. Повторения цикла и рекурсии отображаются возвратными дугами.

Информационный граф содержит вершины операторов, преобразующих или передающих данные. Дуга обозначает возможную прямую информационную связь между их срабатываниями. Проверки условий не включаются, поскольку в принятой модели они только выбирают путь исполнения и не создают числового результата. У компактного графа возможны циклы и петли: они могут связывать разные исполнения одного оператора.

Операционная история строится для заданного входа. Каждому срабатыванию оператора соответствует отдельная вершина, а дуги задают непосредственный последовательный порядок срабатываний. Для завершившегося однопоточного выполнения она является конечной направленной цепью.

Информационная история содержит отдельные вершины срабатываний операторов преобразования данных и только фактически возникшие прямые информационные связи. Она ациклична: значение передаётся от уже выполненного действия последующему. В отличие от компактного информаци-

онного графа, история различает итерации и уровни рекурсии.

Вершины передачи параметров и возврата результата далее включаются как граничные операторы передачи данных. Переход к компактной модели выполняется отождествлением вершин историй, соответствующих одному месту исходного кода.

1.4 Условия параллельного выполнения

Отсутствие пути по данным между двумя срабатываниями позволяет рассматривать их совместное исполнение при выполненных условиях управления. При работе с общей памятью дополнительно проверяются конфликты чтения и записи: нельзя уничтожить ещё не прочитанное значение. Например, два чтения одного числа не конфликтуют.

Информационная история показывает доступный параллелизм вычислений, но сама по себе не разрешает удалять проверки условий. Перестановка действий должна сохранять допустимость арифметики и результат программы. Анализ в четвёртой части относится к исходным последовательным алгоритмам, а не к измеренной многопоточной реализации.

Вывод

Сформулирована задача разворота натурального числа без нулевых цифр. Определены операционное и информационное отношения, а также четыре графовые модели, используемые для сравнения двух алгоритмов.

2 Конструкторская часть

2.1 Нерекурсивный алгоритм

Используется накопитель a, первоначально равный нулю. На каждой итерации выделяются последняя цифра $d = n \bmod 10$ и оставшаяся часть $q = \lfloor n/10 \rfloor$. Цифра присоединяется справа к накопителю: $a \leftarrow 10a + d$. Затем $n \leftarrow q$. При $n = 0$ возвращается накопитель.

На рисунке 1 представлена схема нерекурсивного алгоритма. Парные блоки с идентификатором A обозначают границы цикла; условие находится в начальном блоке. При ложном условии управление переходит за конечную границу цикла.

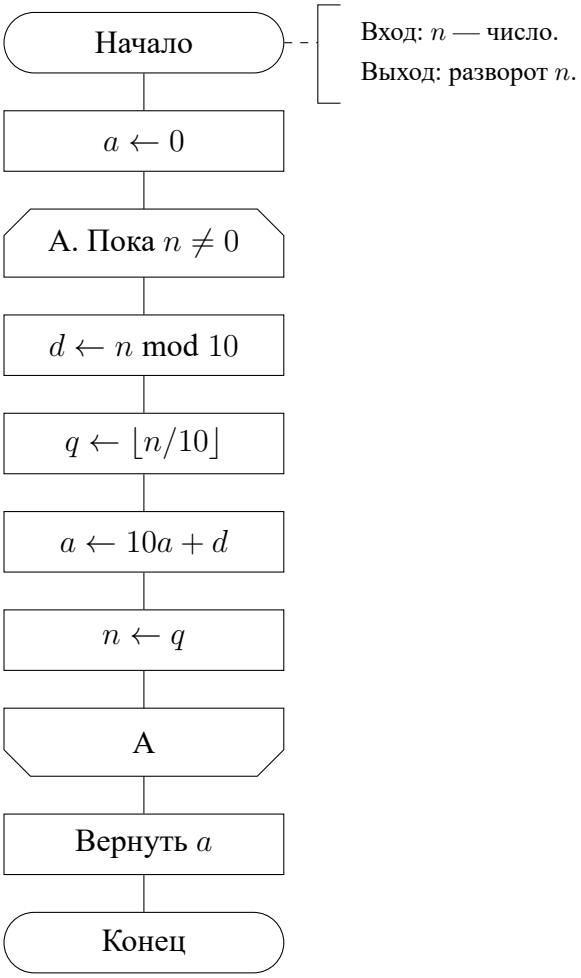


Рисунок 1 – Схема нерекурсивного алгоритма разворота числа

2.2 Рекурсивный алгоритм

Вспомогательная функция $R(n, a)$ принимает ещё не обработанную часть числа и накопленный результат. Она определяется соотношением

$$R(n,a) = \begin{cases} a, & n = 0, \\ R(\lfloor n/10 \rfloor, 10a + n \bmod 10), & n > 0. \end{cases} \tag{2.1}$$

Первый вызов — $R(n, 0)$. Результат дочернего вызова сохраняется в локальной переменной t и возвращается вызывающей функции. Все параметры передаются по значению; разделяемого изменяемого накопителя нет. На рисунке 2 вызов функции показан символом предопределённого процесса.

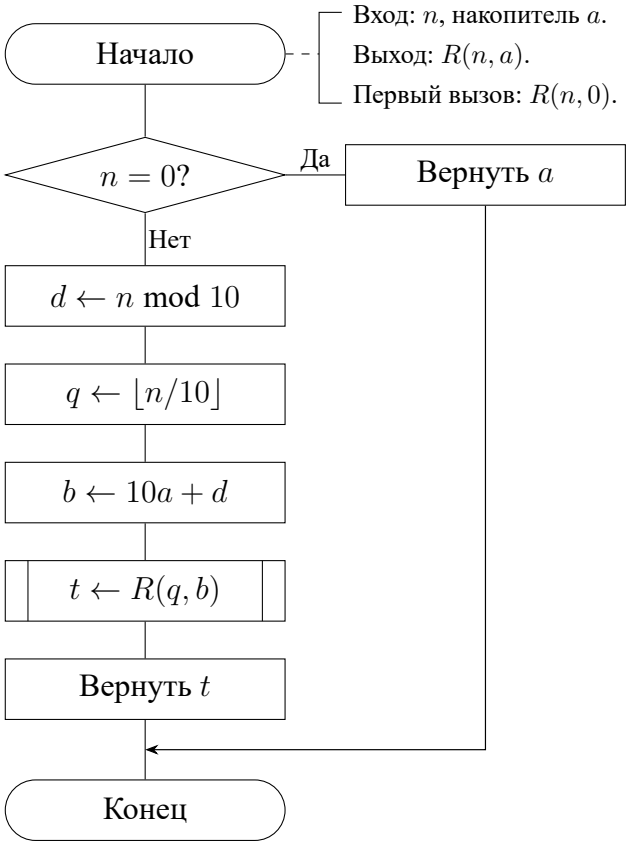


Рисунок 2 – Схема рекурсивного алгоритма разворота числа

2.3 Обоснование корректности и завершения

Пусть исходное число имеет цифры $c_{k-1} \dots c_1 c_0$. После обработки j цифр, $0 \le j \le k$, выполняется инвариант

$$n_j = \left\lfloor \frac{n_0}{10^j} \right\rfloor, \qquad a_j = \sum_{i=0}^{j-1} c_i 10^{j-1-i}, \tag{2.2}$$

где при $j = 0$ сумма равна нулю. В исходном состоянии n_0 не изменено, $a_0 = 0$, поэтому инвариант верен.

При переходе к следующему шагу вычисляется $d = c_j$. Умножение a_j на 10 и прибавление c_j дают запись $c_0 \dots c_j$, а целочисленное деление n_j на 10 удаляет обработанную цифру. Следовательно, инвариант сохраняется.

После k шагов $n_k = 0$, а $a_k = \mathrm{rev}(n_0)$. Итеративный алгоритм возвращает a_k; в рекурсивном алгоритме это значение возвращается из базового случая и без изменения проходит через все ожидающие вызовы.

На каждом небазовом шаге $0 \le \lfloor n/10 \rfloor < n$, поэтому число шагов конечно. Для k цифр выполняются k итераций либо k небазовых и один базовый рекурсивный вызов. В реализации глубина вычислительной рекурсии не превышает 20 вызовов.

Переполнение исключено: $a_j \le 10^j - 1$. При $j < k \le 19$ очередные $10a_j$ и $10a_j + c_j$ не превышают $10^{19} - 1 < 2^{64} - 1$.

2.4 Требования к программе

Программа предоставляет режимы выбора одного или обоих алгоритмов со сравнением результатов. Вводится одно число в строке; пробелы по краям допустимы. Знаки, нулевые цифры, посторонние символы и несколько чисел отклоняются.

Функции возвращают целое число без ввода-вывода и вещественной арифметики. Аргумент проверяется до вычисления: рекурсивно для рекурсивной версии, итеративно для нерекурсивной.

Вывод

Построены схемы двух алгоритмов, доказаны корректность, завершение и отсутствие переполнения.

3 Технологическая часть

3.1 Средства реализации и структура программы

Реализация выполнена на C++17, языке со статической типизацией. Числа представлены типом `std::uint64_t`. Использованы стандартные средства потокового ввода-вывода, обработки исключений и целочисленной арифметики. Исходные файлы программы:

- `algorithms.hpp` — тип числа, граница диапазона и интерфейс функций;
- `algorithms.cpp` — алгоритмы и проверка аргументов;
- `main.cpp` — чтение одной строки, выбор режима и вывод результата;
- `tests/test_algorithms.cpp` — проверки алгоритмов;
- `tests/verify_cli.py` — проверки консольного интерфейса.

Python используется только для запуска проверок интерфейса и подготовки векторных рисунков. Все алгоритмы решения задачи и их проверки по числовому эталону реализованы на C++.

Сборка выполнена компилятором GCC 14.2.0 (MSYS2 UCRT64) в Windows. Основная программа собрана с ключами `-std=c++17 -O2 -Wall -Wextra -Wpedantic`. Для тестов алгоритмов использован также режим `-O0`, при котором проверяется выполнение исходной рекурсии без расчёта на устранение хвостовых вызовов.

3.2 Реализация алгоритмов

В листингах 1 и 2 приведены вычислительные ядра. Обёртки проверяют допустимость аргумента и для рекурсивного ядра передают начальный накопитель, равный нулю. Листинги включаются из фактического исходного файла программы.

Листинг 1 – Нерекурсивное вычислительное ядро

```
Number reverse_iterative_core(Number n) {
    Number a = 0;
    while (n != 0) {
        const Number d = n % 10;
        const Number q = n / 10;
        a = 10 * a + d;
        n = q;
    }
    return a;
}
```

Листинг 2 – Рекурсивное вычислительное ядро

```
Number reverse_recursive_core(Number n, Number a) {
    if (n == 0) {
        return a;
    }
    const Number d = n % 10;
    const Number q = n / 10;
    const Number b = 10 * a + d;
    const Number t = reverse_recursive_core(q, b);
    return t;
}
```

В рекурсивном ядре нет циклов, массивов, строк или списков. Состояние передаётся двумя целочисленными параметрами; локальные d, q, b, t также являются целыми числами. Строковые литералы сообщений об ошибках и имена режимов консольного интерфейса не участвуют в вычислении разворота.

3.3 Тестирование реализаций

Тестирование проведено 26.10.2026. Для каждого допустимого набора проверяются результаты обеих реализаций и свойство двойного разворота $\mathrm{rev}(\mathrm{rev}(n)) = n$. Оно применимо, поскольку нулевые цифры отсутствуют. Ожидаемые значения фиксированных тестов заданы явно; основные случаи представлены в таблице 1.

Таблица 1 – Фиксированные допустимые тесты

Вход	Ожидаемый результат	Проверяемый случай
1	1	Минимальное число
9	9	Одна цифра
12	21	Две разные цифры
179	971	Пример варианта
11111	11111	Одинаковые цифры
12321	12321	Палиндром
123456789	987654321	Возрастающие цифры
987654321	123456789	Убывающие цифры
1234567891234567891	1987654321987654321	19 цифр, непалиндром
9999999999999999999	9999999999999999999	Граница диапазона

Для каждого количества цифр от 1 до 19 дополнительно создаются 1000 чисел. Начальное значение генератора `std::mt19937` равно 20261026; цифра вычисляется как $1 + (g \bmod 9)$. При добавлении очередной цифры d_i слева направо одновременно строятся исходное число и независимый эталон:

$$n_{i+1} = 10n_i + d_i, \qquad e_{i+1} = e_i + d_i 10^i. \tag{3.1}$$

Эталон не вызывает функции разворота. Генератор служит для функционального покрытия; равномерность распределения не предполагается.

Отдельно обе функции должны отклонять 7 недопустимых значений: 0, 10, 101, 1203, 10^{19}, 11111111111111111111 и $2^{64} - 1$. Таким образом проверяются ноль, нулевые цифры и превышение принятой границы. Неверный аргумент приводит к исключению `std::invalid_argument`.

Таблица 2 – Результаты автоматических проверок

Группа	Число случаев	Результат
Фиксированные допустимые числа	10	Пройдены
Генерируемые допустимые числа	19000	Пройдены
Недопустимые аргументы обеих функций	7	Пройдены
Консольный интерфейс	40	Пройдены

Один случай алгоритмического теста проверяет обе реализации; у допустимого случая также проверяется двойной разворот. Число 19017 обозначает количество наборов, а не число вызовов функций.

В 40 проверок интерфейса входят 8 допустимых вводов в отдельных режимах, 30 недопустимых вводов, режим сравнения и неизвестный режим. Проверяются код завершения, результат в стандартном потоке вывода и сообщение в потоке ошибок. Среди ошибок — пустая строка, знаки, дробь, буквы, ведущий ноль, два числа и слишком длинное число. Команды и фактические результаты сохранены в `results/test_results.txt`.

3.4 Пример работы и воспроизведение

Сборка и проверки запускаются командой

Листинг 3 – Запуск сборки и тестов в PowerShell

```
powershell -ExecutionPolicy Bypass -File .\build.ps1
```

Пример сравнения результатов:

Листинг 4 – Пример выполнения программы

```
"179" | .\reverse_number.exe both
iterative: 971
recursive: 971
```

В режиме `recursive` или `iterative` программа выводит только итоговое число. При запуске с клавиатуры ввод завершается клавишей Enter. Успех обозначается кодом 0, недопустимые данные — кодом 1, ошибка выбора режима — кодом 2. Сборка отчёта выполняется XeLaTeX дважды для обновления содержания и ссылок.

Вывод

Оба алгоритма реализованы на C++17. Пройдены 19017 наборов проверок функций и 40 проверок интерфейса, включая граничные значения и ошибочный ввод. Рекурсивное решение соблюдает ограничения варианта U.

4 Исследовательская часть

4.1 Уровень детализации графовых моделей

Модели относятся к вычислительным ядрам листингов 1 и 2. Проверки ввода, диагностика и выбор режима выполняются до них и в графы не включаются. Это позволяет сравнить непосредственно алгоритмы разворота. Каждый оператор присваивания считается одной вершиной; умножение и сложение внутри $10a + d$ не дробятся на машинные команды.

Вершина I_0 обозначает получение параметра n. Соответствие остальных вершин нерекурсивного алгоритма исходному коду приведено в таблице 3.

Таблица 3 – Вершины моделей нерекурсивного алгоритма

Вершина	Оператор / действие
I_0	Получение исходного n
I_1	$a \leftarrow 0$
I_2	Проверка $n \neq 0$
I_3	$d \leftarrow n \bmod 10$
I_4	$q \leftarrow \lfloor n/10 \rfloor$
I_5	$a \leftarrow 10a + d$
I_6	$n \leftarrow q$
I_7	Возврат a

Индекс (j) сверху обозначает номер итерации или глубину вызова, начиная с нуля. Все истории построены для $n = 179$; это конкретные полные выполнения, а не схемы произвольного числа повторений.

4.2 Граф управления нерекурсивного алгоритма

На рисунке 3 показаны все возможные переходы ядра. После I_6 снова проверяется I_2. Дуга «нет» ведёт к возврату; входное число положительно, поэтому тело исполняется хотя бы один раз.

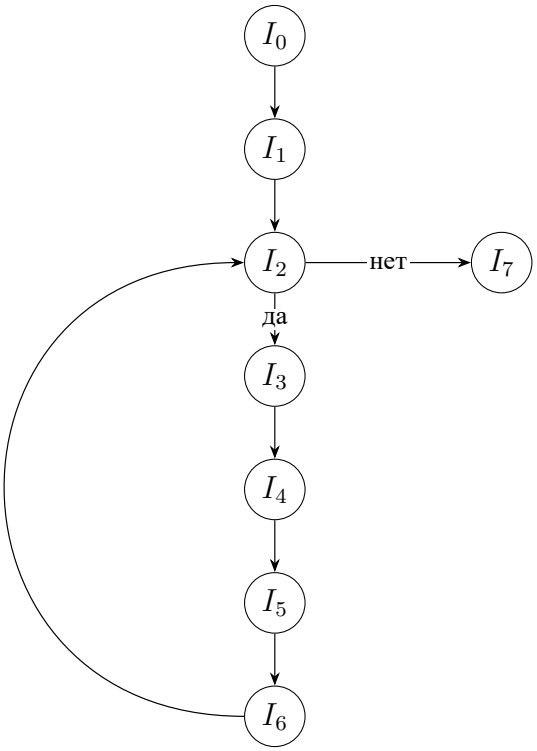


Рисунок 3 – Граф управления нерекурсивного алгоритма

Множество дуг графа:

$$E^I_{\text{у}} = \{(0,1),(1,2),(2,3),(3,4),(4,5),(5,6),(6,2),(2,7)\}.$$

Здесь и далее в множествах дуг префикс вершины опущен.

4.3 Информационный граф нерекурсивного алгоритма

На рисунке 4 нет вершины проверки I_2. Дуги обозначают передачу чисел, а не условия входа в цикл. Начальный n передаётся из I_0 в I_3 и I_4, а на следующих итерациях — из I_6. Петля $I_5 \to I_5$ связывает накопители соседних итераций.

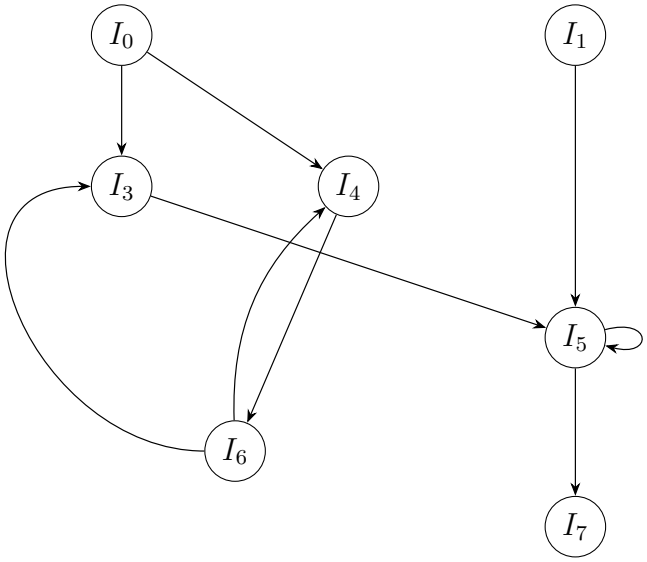


Рисунок 4 – Информационный граф нерекурсивного алгоритма

Множество дуг:

$$E^I_{\text{и}} = \{(0,3),(0,4),(1,5),(3,5),(4,6),$$
$$(5,5),(5,7),(6,3),(6,4)\}.$$

Дуги $I_3 \to I_4$ нет: выделение последней цифры не вырабатывает операнд для деления. Оба оператора читают одно и то же значение n.

4.4 Операционная история нерекурсивного алгоритма

Состояния после итераций приведены в таблице 4. Они одинаковы у обеих реализаций.

Таблица 4 – Изменение состояния для числа 179

j	n_j	a_j	d_j	q_j
0	179	0	9	17
1	17	9	7	1
2	1	97	1	0
3	0	971	—	—

На рисунке 5 показана полная цепь из 19 срабатываний: два начальных, три группы по пять срабатываний, последняя ложная проверка и возврат. Цепь читается по стрелкам, попеременно слева направо и справа налево.

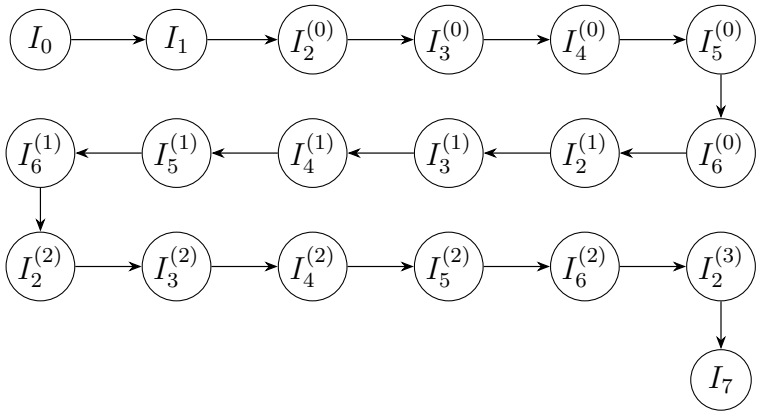


Рисунок 5 – Операционная история нерекурсивного алгоритма при $n = 179$

Значения проверок $I_2^{(0)}$, $I_2^{(1)}$, $I_2^{(2)}$ истинны, $I_2^{(3)}$ — ложно. Порядок $I_3^{(j)} \to I_4^{(j)}$ присутствует в операционной истории, хотя прямой зависимости по данным между этими срабатываниями нет.

4.5 Информационная история нерекурсивного алгоритма

На рисунке 6 представлены 15 вершин преобразований данных и 16 прямых информационных дуг. Контрольные проверки отсутствуют. Петля компактной модели раскрывается в цепочку $I_5^{(0)} \to I_5^{(1)} \to I_5^{(2)}$.

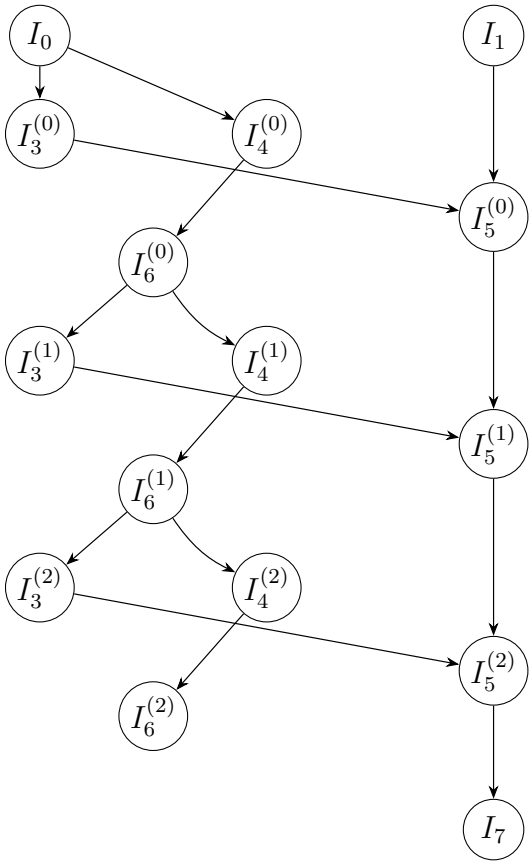


Рисунок 6 – Информационная история нерекурсивного алгоритма, $n = 179$

Для каждого $j = 0, 1, 2$ источник n_j соединён с $I_3^{(j)}$ и $I_4^{(j)}$; это I_0 при $j = 0$ и $I_6^{(j-1)}$ при $j > 0$. Также присутствуют дуги $I_3^{(j)} \to I_5^{(j)}$ и $I_4^{(j)} \to I_6^{(j)}$. Источником a_j для $I_5^{(j)}$ служит I_1 либо $I_5^{(j-1)}$. Последняя дуга $I_5^{(2)} \to I_7$ передаёт 971.

4.6 Вершины моделей рекурсивного алгоритма

Соответствие вершин листингу 2 дано в таблице 5. Вызов в строке присваивания t разделён на две фазы, поскольку между ними выполняется дочерняя функция.

Таблица 5 – Вершины моделей рекурсивного алгоритма

Вершина	Оператор / действие
R_0	Получение параметров (n, a) текущего вызова
R_1	Проверка $n = 0$
R_2	Возврат a в базовом случае
R_3	$d \leftarrow n \bmod 10$
R_4	$q \leftarrow \lfloor n/10 \rfloor$
R_5	$b \leftarrow 10a + d$
R_{6a}	Передача аргументов (q, b) дочернему вызову
R_{6b}	Получение результата дочернего вызова в t
R_7	Возврат t из текущего вызова

Важное различие: R_{6a} использует q и b, а R_{6b} использует возвращённое дочерней функцией число. Поэтому переносить входные зависимости R_{6a} на R_{6b} нельзя. Локальные переменные разных глубин являются разными объектами, даже если их имена совпадают.

Для $n = 179$ параметры вызовов равны $(179, 0)$, $(17, 9)$, $(1, 97)$ и $(0, 971)$. Глубины обозначены $j = 0, 1, 2, 3$. Все возвращаемые значения равны 971.

4.7 Граф управления рекурсивного алгоритма

На рисунке 7 дуга $R_{6a} \to R_0$ означает вход в дочернюю функцию. Возврат из неё приводит к R_{6b} ожидающего родителя. При возврате из исходного вызова управление передаётся внешней обёртке; на графе её операторы не изображены.

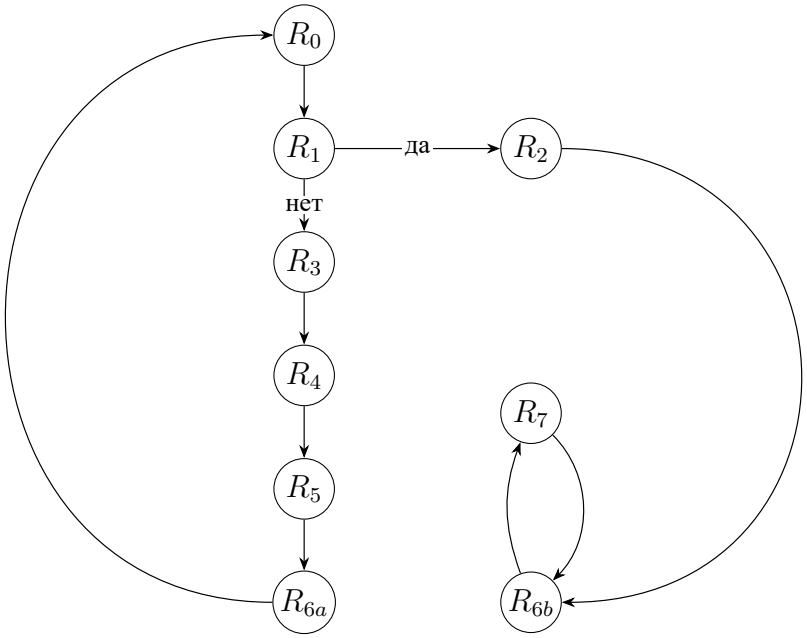


Рисунок 7 – Граф управления рекурсивного алгоритма

Множество внутренних дуг:

$$E_{\text{у}}^{R} = \{(0,1),(1,2),(1,3),(3,4),(4,5),(5,6a),$$
$$(6a,0),(2,6b),(6b,7),(7,6b)\}.$$

Дуги возврата контекстны: переход разрешён только в продолжение соответствующего вызова. Компактный граф без стека не хранит номер ожидающего родителя; его восстанавливают стеком вызовов, а в истории явно обозначают глубиной.

4.8 Информационный граф рекурсивного алгоритма

На рисунке 8 показана передача данных. Из параметров вычисляются d, q, b, а в базовом случае возвращается a. Возвращаемое дочерним вызовом значение поступает в R_{6b} родителя из R_2 или R_7 дочернего уровня.

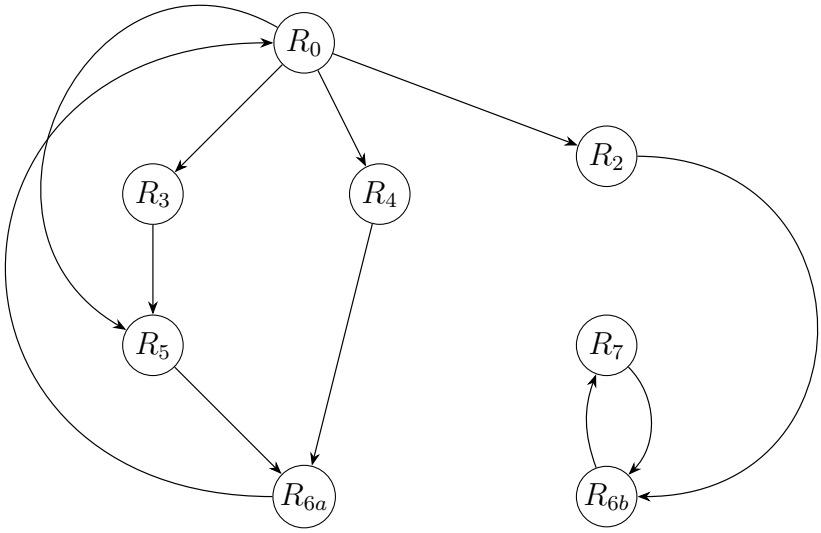


Рисунок 8 – Информационный граф рекурсивного алгоритма

Множество дуг:

$$E^R_{\text{и}} = \{(0,2),(0,3),(0,4),(0,5),(3,5),(4,6a),\\ (5,6a),(6a,0),(2,6b),(6b,7),(7,6b)\}.$$

Дуг $R_0 \to R_{6b}$ и $R_{6a} \to R_{6b}$ нет: получение ответа не использует исходные параметры как числовые операнды. Сам факт ожидания возврата относится к управлению. Цикл $R_{6b} \to R_7 \to R_{6b}$ связывает разные глубины, а не одновременные действия одного вызова.

4.9 Операционная история рекурсивного алгоритма

На рисунке 9 приведены 27 срабатываний. Для глубин 0–2 выполняются по шесть действий спуска: $R_0, R_1, R_3, R_4, R_5, R_{6a}$. На глубине 3 выполняются R_0, R_1, R_2, затем для глубин 2, 1 и 0 — по два действия R_{6b}, R_7.

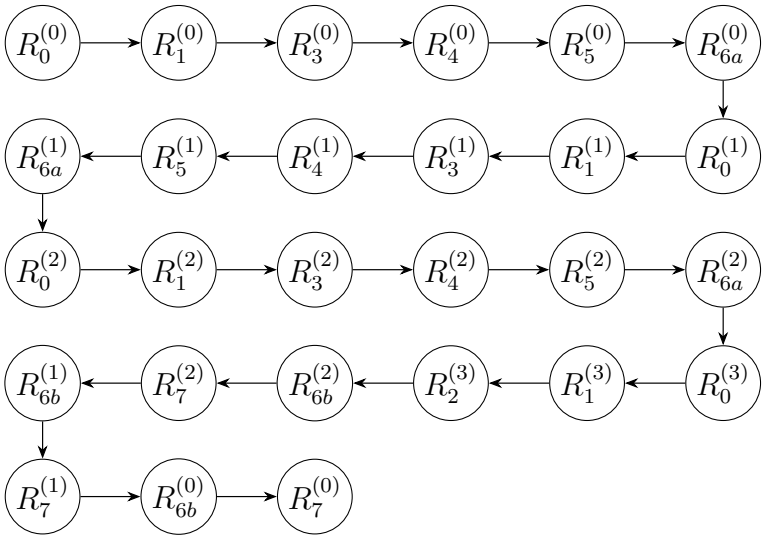


Рисунок 9 – Операционная история рекурсивного алгоритма при $n = 179$

Нельзя соединять $R_{6a}^{(j)}$ непосредственно с $R_{6b}^{(j)}$ в операционной истории: между ними исполняется весь дочерний вызов. В частности, $R_2^{(3)}$ непосредственно предшествует $R_{6b}^{(2)}$, а $R_7^{(2)}$ — $R_{6b}^{(1)}$.

4.10 Информационная история рекурсивного алгоритма

На рисунке 10 показаны 23 вершины и 28 дуг. Левая часть соответствует вычислению очередной цифры и накопителя, средняя — передаче аргументов, правая — возврату одного и того же результата 971.

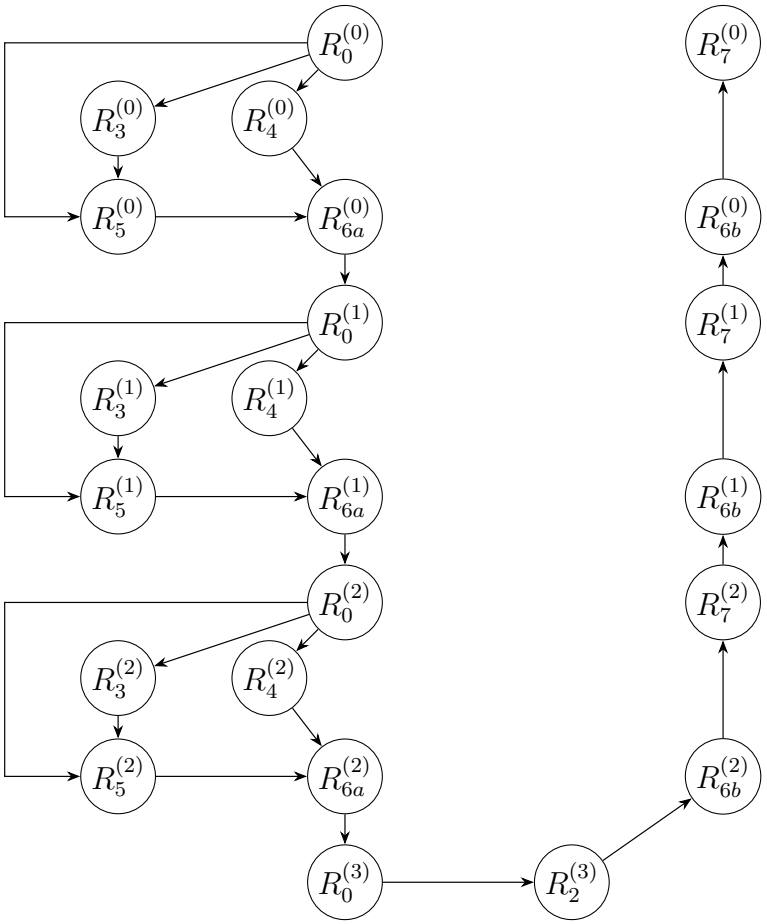


Рисунок 10 – Информационная история рекурсивного алгоритма, $n = 179$

Для $j = 0, 1, 2$ дуга $R_{6a}^{(j)} \to R_0^{(j+1)}$ передаёт новую пару параметров. Цепь возврата начинается дугой $R_0^{(3)} \to R_2^{(3)}$, затем проходит через $R_{6b}^{(2)}, R_7^{(2)}, R_{6b}^{(1)}, R_7^{(1)}, R_{6b}^{(0)}, R_7^{(0)}$. Исходный параметр родителя не является непосредственным источником значения t.

В информационных историях нет стрелок от проверок условий и нет стрелок, добавленных только ради воспроизведения текстового порядка. Все 10 рисунков отчёта заданы средствами TikZ и включены как векторные объекты, без растрового преобразования. Файлы моделей и проверяемые множества дуг находятся в `figures/` и `results/graph_models.json`.

4.11 Возможности параллельного выполнения исходного кода

В нерекурсивном ядре строки `const Number d = n % 10;` и `const Number q = n / 10;` могут выполняться параллельно после проверки условия: обе читают одно значение n, записывают разные локальные переменные и не зависят друг от друга. Вычисление `a = 10 * a + d;` должно ждать d, а `n = q;` — q и завершения чтений старого n.

После вычисления d и q присваивания `a = 10 * a + d;` и `n = q;` также независимы: они записывают разные объекты и не читают результаты друг друга. Нельзя начинать новую итерацию целиком до подготовки требуемых ею значений n и a. Цепочки обновлений n_j и особенно a_j сохраняют последовательные зависимости.

В рекурсивном ядре строки вычисления d и q также могут выполняться параллельно. После получения d вычисление b может перекрываться с ещё не завершённым вычислением q, так как b использует a и d. Передача аргументов дочернему вызову требует готовности и q, и b.

Соседние рекурсивные вызовы нельзя запускать как независимые задачи: параметры следующего вызова ещё должны быть вычислены текущим. Получение t и возврат из родителя ожидают результат потомка. У рекурсивного алгоритма одна ветвь вычислений, поэтому здесь нет независимых рекурсивных подзадач, подобных двум ветвям деления диапазона пополам.

Точное соответствие фрагментов и ограничений сведено в таблицу 6. Перечислен потенциальный параллелизм; в представленной программе потоки не создаются.

Таблица 6 – Независимые действия и зависимости

Фрагмент	Обоснование
$I_3^{(j)}$ и $I_4^{(j)}$	Общий вход n_j, разные выходы; допустимо совместное выполнение
$I_5^{(j)}$ и $I_6^{(j)}$	Независимы после получения d_j, q_j и чтения старого n_j
$I_5^{(j)}$ и $I_5^{(j+1)}$	Последовательны: второй использует новый накопитель первого
$R_3^{(j)}$ и $R_4^{(j)}$	Независимые остаток и частное от одного параметра
$R_4^{(j)}$ и $R_5^{(j)}$	Независимы при готовом d_j; вызов ждёт оба результата
$R_{6a}^{(j)}$ и вход потомка	Передача данных: потомку нужны вычисленные аргументы
Возвраты соседних глубин	Родитель получает результат только от завершившегося потомка

Для нескольких независимых исходных чисел допустима параллельная обработка целых задач с раздельными локальными состояниями. Допустимо также одновременное выполнение двух реализаций для одного входа при раздельных накопителях; это ускоряет только проверку двух ответов и не устраняет зависимости внутри каждого алгоритма. Вывод в общий поток следует выполнять после получения результатов.

Если заранее сохранить неизменяемые n_j, d_j и отдельные накопители, часть операций соседних итераций можно перекрыть. Это уже преобразование организации хранения исходной программы, а не разрешение одновременно изменять её единственный a. Кроме того, цепочка $a_{j+1} = 10a_j + d_j$ в рассматриваемой форме сохраняется.

4.12 Сравнение графовых моделей и ресурсов

Пусть k — число десятичных цифр. При выбранной детализации операционная история нерекурсивного ядра содержит $5k+4$ вершины, информационная — $4k+3$. Для рекурсивного ядра соответствующие количества равны $8k+3$ и $7k+2$. Эти величины учитывают передачу параметров и возврат как отдельные действия и не являются количеством машинных инструкций.

У обеих реализаций число арифметических шагов пропорционально k. Информационные истории содержат цепочку из k обновлений накопителя, поэтому длина критического пути имеет порядок $\Theta(k)$ даже при независимом вычислении частного и остатка. Доступный параллелизм на один шаг ограничен постоянным числом действий; роста числа независимых ветвей с k нет.

Таблица 7 – Сопоставление реализаций

Свойство	Нерекурсивная	Рекурсивная
Организация повторений	Цикл	Вызовы функции
Время ядра при k цифрах	$\Theta(k)$	$\Theta(k)$
Дополнительная память ядра	$\Theta(1)$	$O(k)$
Операционная история, $n = 179$	19 вершин	27 вершин
Информационная история, $n = 179$	15 вершин	23 вершины
Прямые информационные дуги, $n = 179$	16	28
Критический путь	$\Theta(k)$	$\Theta(k)$
Циклы и массивы в ядре	Один цикл	Отсутствуют

Оценка памяти рекурсии предполагает обычный стек вызовов. Рекурсия хвостовая, поэтому компилятор может устранить часть или все кадры, но программа не полагается на эту оптимизацию. Итеративная проверка аргумента требует $O(1)$ памяти, рекурсивная — $O(k)$; таким образом, обёртки сохраняют приведённые порядки памяти полных функций. Проверка цифр добавляет ещё один линейный проход по цифрам.

Здесь $\Theta(k)$ рассматривается в модели, где элементарные операции с машинным целым имеют постоянную стоимость. Фактический диапазон реализации ограничен $k \leq 19$. Замеры времени не требуются поставленной целью и в работе не выполнялись; преимущество по времени конкретного исполняемого файла из числа вершин не выводится.

Вывод

Для каждого алгоритма построены четыре графовые модели. Анализ исходного кода выявил независимое вычисление частного и остатка и отдельные независимые присваивания. Цепочка накопителя и передача результатов рекурсии ограничивают параллелизм; длина критического пути у обоих алгоритмов линейна по числу цифр.

ЗАКЛЮЧЕНИЕ

Проведено сравнение рекурсивного и нерекурсивного алгоритмов разворота числа для варианта U на основе графовых моделей.

Даны определения операционного и информационного отношений и четырёх графовых моделей. Разработаны два последовательных алгоритма, построены их схемы, доказаны корректность и завершение. Реализации на C++17 возвращают целое число; рекурсивная версия не использует циклы, строки и массивы при вычислении результата.

Тестирование включило 19010 допустимых и 7 недопустимых наборов аргументов для обеих реализаций, а также 40 проверок интерфейса. Все проверки завершились успешно. Для каждого алгоритма построены граф управления, информационный граф, операционная и информационная истории выполнения при $n = 179$. Передача аргументов рекурсивного вызова и получение результата отражены разными вершинами.

Установлено, что частное и остаток от деления одного числа на 10 могут вычисляться независимо. Обновления накопителя между шагами и возвраты результатов между уровнями рекурсии должны сохранять порядок зависимостей. Оба вычислительных ядра имеют трудоёмкость $\Theta(k)$; нерекурсивное использует постоянную дополнительную память, рекурсивное в обычной стековой реализации — линейную. Цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — Введ. 2018-07-01.

2. ГОСТ 19.701–90 (ИСО 5807–85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. — Введ. 1992-01-01.

3. Воеводин В. В., Воеводин Вл. В. Параллельные вычисления. — СПб.: БХВ-Петербург, 2002. — 608 с.

4. Параллельная обработка данных: программа курса // Информационно-аналитический центр по параллельным вычислениям PARALLEL.RU. — URL: https://parallel.ru/info/parallel/parallel_programme.html (дата обращения: 26.10.2026).